

■ AQUAGUARD

Project by

Sanskriti Patel — Grade 8

Pramveer Kumar — Grade 9

Agrima Jin — Grade 8

Dhanvith J Patel — Grade 7

Community Water Quality & Usage Monitor

Hack-AI-Thon 2026 | SDG 6 + SDG 12

PROJECT DOCUMENTATION

1. Project Overview

AquaGuard is a Python-based water monitoring system designed for households, schools, and communities. It records daily water consumption and basic water-quality observations such as pH and turbidity. The system checks these values against project-defined demonstration thresholds, detects abnormal conditions, generates alerts and recommendations, calculates consumption trends, and produces a monitoring report.

The project is primarily implemented in Python and demonstrates Python fundamentals and Object-Oriented Programming (OOP), as required by the Hack-AI-Thon problem statement.

2. Problem Statement

Water wastage and poor awareness of water-quality conditions can create problems for households, schools, and communities. Excessive consumption may indicate wastage or leaks, while unusual pH or turbidity readings may indicate that a water sample needs further attention.

AquaGuard provides a simple way to record these observations and convert them into useful information such as quality status, alerts, trends, and conservation recommendations.

3. SDG Alignment

SDG 6 – Clean Water and Sanitation

AquaGuard supports SDG 6 by monitoring basic water-quality observations and flagging potentially unsafe sample conditions based on pH and turbidity.

SDG 12 – Responsible Consumption and Production

AquaGuard supports SDG 12 by monitoring daily water consumption, identifying excessive use, and encouraging responsible water use and conservation.

4. Objectives

Record daily water consumption, pH, and turbidity.

Represent monitored locations and users using OOP classes.

Identify abnormal readings using selected project thresholds.

Generate understandable alerts for abnormal conditions.

Calculate a simple consumption trend from recorded readings.

Provide practical water-conservation recommendations.

Generate a location-wise monitoring report.

5. Scope of the Prototype

The current prototype focuses on a simple console-based Python implementation.

Input is represented by sample data inside the Python program.

The prototype monitors one location in the demonstration.

The system uses pH, turbidity, and daily consumption.

The report is displayed in the console.

No database, IoT sensor, web API, login system, GUI, SMS/email service, or cloud service is required by the current problem statement.

6. System Design

The system follows a simple object-oriented structure:

Location – stores the place being monitored.

User – stores the person recording the readings.

WaterReading – stores one day's consumption and water-quality data and checks abnormal values.

MonitoringReport – collects readings and generates trends, alerts, recommendations, and the final report.

Relationship Between Classes

A Location represents where monitoring takes place. A User represents who records the data. Each WaterReading belongs to a Location and is recorded by a User. A MonitoringReport belongs to a Location and contains multiple WaterReading objects.

7. OOP Concepts Used

Classes and Objects

The project creates objects from Location, User, WaterReading, and MonitoringReport classes.

Encapsulation

Data and related operations are grouped inside classes. For example, WaterReading contains the data and methods used to evaluate that reading.

Methods

Methods such as `is_abnormal()`, `get_alerts()`, `quality_status()`, `average_consumption()`, and `consumption_trend()` perform specific tasks.

Composition / Object Relationships

MonitoringReport stores WaterReading objects, while each reading refers to a Location and User.

Docstrings

Important classes and methods include documentation strings to explain their purpose.

8. Data and Project-Defined Thresholds

The Hack-AI-Thon problem statement requires defined thresholds but does not provide numerical threshold values. Therefore, the following values are selected by the project for demonstration.

Important: These are project-defined demonstration values, not numerical thresholds supplied by the Hack-AI-Thon problem statement.

9. Algorithms and Logic

9.1 Abnormal Reading Detection

Check whether pH is below 6.5 or above 8.5.

Check whether turbidity is greater than 5.0 NTU.

Check whether daily consumption is greater than 500 L.

If any condition is true, mark the reading as abnormal.

Generate a specific alert for each abnormal condition.

9.2 Water Quality Status

The prototype marks a reading as Unsafe when pH is outside the selected pH range or turbidity is above the selected turbidity limit. High consumption creates a usage alert but does not by itself make the water quality Unsafe.

9.3 Average Consumption

Average consumption is calculated as:

Average = Total consumption of all readings ÷ Number of readings

9.4 Consumption Trend

The prototype compares the first and last consumption values. If the last value is more than 15% higher than the first, the trend is reported as Increasing. If it is more than 15% lower, the trend is Decreasing. Otherwise, it is Stable. With fewer than two readings, the result is Not enough data.

10. Input–Output Flow

Create Location object.

Create User object.

Create WaterReading objects using sample data.

Add readings to MonitoringReport.

Check pH, turbidity, and consumption.

Generate alerts and quality status.

Calculate average consumption and trend.

Generate recommendations.

Print the complete monitoring report.

11. Sample Test Data

12. Expected Demonstration Result

Average consumption: approximately 426.7 L/day.

Consumption trend: Increasing (possible wastage).

Overall report status: ATTENTION NEEDED.

Alerts: low pH, high turbidity, and high consumption.

Recommendations: reduce water use/check for leaks, investigate rising consumption, consider filtration or further testing for high turbidity, and get abnormal pH readings tested.

13. Testing

Test Case 1 – Normal Reading

Input: 320 L/day, pH 7.1, turbidity 2.3 NTU. Expected result: Safe quality status and no abnormal alert.

Test Case 2 – Unsafe Quality Reading

Input: 450 L/day, pH 6.2, turbidity 6.8 NTU. Expected result: Unsafe quality status with low-pH and high-turbidity alerts.

Test Case 3 – Excessive Consumption

Input: 510 L/day, pH 7.4, turbidity 3.1 NTU. Expected result: Safe quality status but a high-consumption alert.

Test Case 4 – Trend

Using 320 L, 450 L, and 510 L in chronological order should produce an Increasing trend.

14. Limitations

The current prototype uses sample data rather than live sensor measurements.

The system uses project-defined demonstration thresholds.

The trend calculation is a simple first-versus-last comparison.

The current report is console-based.

The prototype does not store data permanently in a database.

A water-quality alert is an indication for attention, not a laboratory or medical certification that water is safe to drink.

15. Future Scope

Connect real water-quality sensors.

Add database storage for long-term records.

Support multiple households, schools, or community locations.

Add graphical dashboards and charts.

Add automatic data import from sensors or files.

Add exportable PDF/CSV reports.

Add optional notification systems for important alerts.

16. Requirements and Execution

Environment:

Python 3.x

Standard Python libraries used by the prototype: datetime and typing.

No external package or API is required for the current console version.

Run command:

```
python aquaguard.py
```

If the source file has a different name, run Python followed by that filename. The program creates the sample objects and prints the monitoring report to the console.

17. Conclusion

AquaGuard is a simple but functional Python prototype that directly addresses the Hack-AI-Thon challenge. It demonstrates the required Location/User, WaterReading, and MonitoringReport classes; records water-use and quality observations; detects abnormal conditions; generates alerts and recommendations; calculates consumption trends; and produces a location-wise monitoring report.

The project also connects the technical solution to SDG 6 through water-quality awareness and SDG 12 through responsible water consumption and conservation.

18. Reference to the Hack-AI-Thon Requirement

The official project prompt requires a Python-based water monitoring system for households, schools, or communities, with Location/User, WaterReading, and MonitoringReport classes; recording of consumption, pH, and turbidity; abnormal-reading detection; alerts/recommendations; and location-wise or household-wise summaries. It also requires documentation covering the problem statement, SDG mapping, objectives, system design, classes/OOP concepts, algorithms/logic, input-output flow, testing, limitations, and future scope.